Code and Test Generation for I4.0 State Machines with LLM-based Diagram Recognition

Björn Otto*, Assanali Aidarkhan†, Marko Ristin†, Nico Braunisch§,
Christian Diedrich*, Hans Wernher van de Venn‡, Martin Wollschlaeger§

*Institute for Automation and Communication, Otto von Guericke University {bjoern.otto, christian.diedrich}@ifak.eu
†Department of Computer Science, Nazarbayev University, Astana, Kazakhstan, {assanali.aidarkhan, marko.ristin}@nu.edu.kz
‡Institute of Mechatronic Systems, Zurich University of Applied Sciences (ZHAW), Zurich, Switzerland {vhns@zhaw.ch}
§Institute of Applied Computer Science, TU Dresden, Dresden, Germany
{nico.braunisch, martin.wollschlaeger}@tu-dresden.de

2025 IEEE 21st International Conference on Factory Communication Systems (WFCS) | 979-8-3315-3005-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/WFCS63373.2025.11077624

***Abstract*—In the context of Industry 4.0, the automatic code and test generation from state diagrams embedded in specifications is a critical challenge for software correctness. In this paper we present an approach that leverages Large Language Models (LLMs) for the recognition of state diagrams to generate code and unit tests automatically. We compare the performance of LLMs with traditional computer vision models, highlighting the advantages of LLMs in terms of generalization and simplicity of setup. The results on two prominent industrial communication protocols, PROFINET and OPC UA, demonstrate the applicability of the approach, achieving significant reductions in manual effort and improving the accuracy of code and test generation.**

***Index Terms*—Code generation, Large Language Model, LLM, Test case generation, Diagram recognition, Industry 4.0**

I. Introduction

Manufacturing and industrial automation has undergone several evolutions, with the latest transformation – Industry 4.0 – marking the digitization of the entire production process. This shift enables highly customizable value chains that, in turn, demand a high degree of interoperability. Interoperability plays a crucial role by allowing production processes to seamlessly exchange data, even across different manufacturers. However, achieving this level of connectivity requires manufacturers to align on common standards. These are described in detail in corresponding specifications. Manufacturers then have to implement these specifications correctly to be interoperable. However, this is hampered by the fact that specifications are often very extensive and complex. Consequently, a significant amount of effort and time is spent on implementing these specifications. Hence reducing this effort is of major importance, especially in the dynamic and rapidly changing Industry 4.0.

In the context of communication protocols, state machines have shown to be a helpful abstraction for describing system behavior. Many specifications also include rendered images of these state machines in the form of state diagrams. Figure 1 shows an example of such a state diagram.

Developers spend a significant amount of time to convert these state diagrams into code. If state diagrams are given in formalized, machine-readable form, converting them to code is simple. Unfortunately, state diagrams are rarely given in such a structured form. Most of the time, the specification is proprietary and bought in form of a fixed PDF document, where diagrams are embedded as images.

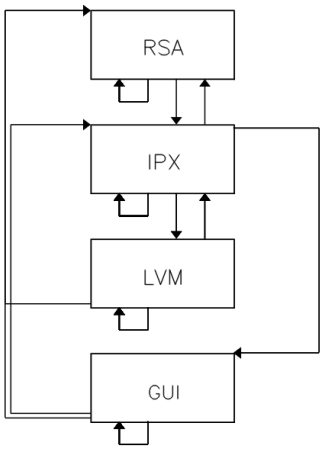


Fig. 1: State diagram from the IEC 61158 standard [1]

This leads to the problem that if you want to implement or test the specification which includes the state diagrams, you have to write the code for the state machines manually. This is tedious. In particular, it is hard to double-check the implementation: the proof-reader needs to be proficient in programming enough to read and understand the code.

In this work, we present an approach to significantly reduce this manual effort. The core idea is to utilize diagram recognition to extract the state diagrams from the specification and make them machine-readable. Based on the machine-readable form, our approach is able to generate boilerplate code, saving the developers a significant coding effort. In addition, we leverage the formalized state diagrams to derive unit tests automatically, which facilitates the development even more, and improves the testability (and, consequently, the correctness) of the final implementations. Figure 2 shows our pipeline.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.

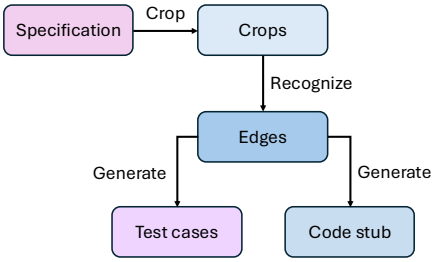


Fig. 2: Overview of our pipeline

LLMs versus Computer Vision

There are basically two possible approaches to diagram recognition. One is to use computer vision [2]–[4], where models are specifically trained to recognize image pixels as the individual diagram elements such as rectangles, text, and arrows. The individual models are finally combined into a holistic representation of the diagram for further processing.

The other approach is to use large language models (LLMs) which are trained as a sequence-to-sequence translation models, usually for question & answering [4]. The model is trained to translate a question (a.k.a. *prompt*) into an answer. The input and the output are represented with *tokens*, *i.e.*, predefined enumerations of words and sub-word parts. This representation can also capture images [5], where the model learns to translate an image to its caption, using cross-modality learning. Since there is only one model to capture all the training data, this one model profits from transfer learning effects coming from different domains. This makes the LLMs a *general* question answering machine.

Due to this capability to generalize, the LLMs can be considered the most relevant engine behind the current AI trend, with many big companies having strong direct and indirect interests in their development. This leads to abundance of training data which also makes them versatile: they can answer arbitrary questions, including the diagram recognition.

On the other hand, the computer vision models are trained on much smaller data sets. As they need to be custom-tailored for a specific problem, there is less interest from the stakeholders. They attract attention only from the parties which are directly involved in diagram recognition, missing out on the transfer learning effects which the LLMs hugely profit from. Hence, whenever a new style of diagrams need to be recognized, we need to manually annotate the diagrams so that the computer vision models can be fine-tuned on this new style. This involves a setup of annotation tools, manual re-tracing and annotation of sufficient number of diagrams, re-training the models which might also include the fine-tuning of hyper-parameters *etc*. In contrast, as we show in Subsection III-A, LLMs require prompt engineering, *i.e.*, adapting the question given as text so that the model outputs the correct answer in the expected format. Given the generalizability of LLMs and the simplicity of their setup, we focus our work on how LLMs can be used for recognition of state diagrams.

II. Related Work

A. Diagram Recognition

1) Computer Vision: Computer vision in its strict sense, *i.e.*, machine learning models trained and applied on image modality, has been long used for recognizing different types of diagrams [6]. Usually, the system consists of different models which are combined: a model for Optical Character Recognition (OCR) to capture the text, line detectors for edges, object detector for shapes and arrow heads, image classification for fine-grained classification of symbols *etc*. The individual models are trained and applied in separation, and their results are combined programmatically. Such systems have been successfully applied to engineering diagrams [7]–[9], software class diagrams [10]–[12], general hand-written diagrams [13], business process diagrams [14], including state diagrams [15]. The computer vision-based systems have two serious drawbacks:

Complexity The fragmentation across models demands specialized training and evaluation, with diverse techniques like image classification, line detection, and OCR, each requiring unique metrics (*e.g.*, mean average precision or accuracy [16]). Combining results into a cohesive diagram remains a significant challenge, limiting prior state diagram recognition methods [15].

Generalization Models can only recognize elements seen during training, but training sets are limited since they are tailored to niche applications [6]. Re-training for new diagram types is impractical for end-users, who often lack the expertise and infrastructure for data annotation and model updates.

In light of these drawbacks, we base our recognition on large language models, which is elaborated next.

2) Diagram Recognition with LLMs: In recent years, LLMs have become widely popular [4]. At their core is a transformer neural network with an attention mechanism, enabling context-aware sequence translation, such as turning a question into an answer. Transformers can be adapted for other modalities, such as images, by encoding image patches as sequence elements [5]. Unlike custom-tailored computer vision systems, LLMs like ChatGPT [17], Claude [17], or Llama 3 [18] are foundational models used by a large user group for a diverse set of tasks. They are therefore trained on vast datasets, enabling exceptional generalization [19]. In Industry 4.0, LLMs have been used for tasks like generating control code [20], [21], extracting structutred information from product descriptions [22], including recognition of diagrams such as piping and instrumentation diagrams (P&ID) [23] or software class diagrams [24]. To the best of our knowledge, no prior work has focused on state diagrams. Unlike other engineering and class diagrams, state diagrams (Figure 1) feature distinct nodes and edges with clear semantics (state transitions), and significantly more edge crossings than class diagrams. Unlike prior work [23], which combines recognition and generation in a single step, our approach splits them in two (recognition followed by generation). As shown in Subsubsection IV-F2, this two-step split offers advantages.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.

B. Code Generation

1) Model-based: Model-based code generation is a well-established field and has also been applied to Industrie 4.0 applications [25], [26]. In this context, the term *model* refers to a system model, *i.e.*, any kind of formal description of the desired system behavior. For example, the authors of [26] use a subset of the Python programming language to model data structures of Digital Twins and leverage this formalization to generate SDKs in various languages.

2) LLM-based: LLM-based code generation has been studied extensively, especially due to the breakthroughs reached by recent LLMs like ChatGPT [17]. In [27] the authors evaluate how useful LLMs are for software development. The authors conclude that LLMs are in particular helpful when generating foundational code structures. The authors of [28] instantiate multiple LLM-based agents to mimic typical roles in the software development process. They find LLM based code generation to generate more correct code with less code smells.

C. Test Generation

1) Model-based: Model-based test generation has been the focus of considerable research over the past decades [29], [30]. Model-based testing relies on a system model to derive test cases. This may be the same model as used during development or even for code generation as described in the previous subsection. It may also rely on a dedicated test model to derive test cases. Model-based testing often leverages coverage measures to generate test cases. For example, given a state-machine, a model-based testing approach can aim to cover all nodes or edges in the state-machine when generating test cases. In [31] the authors use OpenAPI descriptions to generate test cases for digital twins automatically. They highlight the improved efficiency and coverage of automated test case generation in comparison to manual test case definition.

2) LLM-based: LLM-based test case generation relies on an existing code base to derive test cases automatically. By analyzing the code base an LLM is instructed to generate a set of tests. In [32] the authors present *Chatunitest*, a tool to derive unit tests automaitcally. *Chatunitest* is able to cover more lines than other popular traditional or LLM-based tools. Another tool, *SymPrompt* [33], offers a multi-stage approach for test case generation where each phase is fed with information about execution paths of the previously generated tests. This way, the authors are able to improve coverage compared to single-pass solutions.

III. Methodology

In this study, we explore the performance of LLMs in inferring the nodes and connecting edges from diagrams embedded in PDF files. The diagrams are incorporated in PDF documents as images, and so can not be directly extracted for further usage. Hence, in our work, we apply a three-step process: (i) Cropping; (ii) Recognition; and (iii) Generation. The implementation of our method is publicly available[1].

[1]https://zenodo.org/records/14730727

Cropping The initial step involves cropping from PDF documents. Since the diagrams are embedded as images alongside the text, we manually crop the relevant images from the surrounding content. The manual labeling approach ensures high precision, but it is time-intensive. Although automatic cropping approaches exist, they still require significant improvements in precision and adaptability for diverse documents [34], so this work focuses on recognition, leaving comprehensive cropping methods for future research.

Recognition After the diagrams have been cropped, the next step is to interpret the information that they contain. We employ the LLMs for a recognizing task by providing a carefully designed prompt along with the input diagram. The LLM is expected to recognize the state diagram and output it in the desired machine-readable format such that it can be further processed (cf. Subsection III-A).

Code and Test Generation Since the LLM is instructed to output the recognized diagram in a machine-readable format, the response of the LLM is programmatically parsed into an object-oriented representation. Based on this representation, we procedurally generate the implementation stubs as well as the test code.

In the following subsections, we explain in more detail how the LLMs generate response for inferring edges in the input state diagram. We explain how we engineered the prompt, explore two different settings with varying number of examples given to LLMs (zero-shot and few-shot), and look into how to use the LLM response for code generation.

A. Prompt Engineering

Effective prompt engineering is crucial for guiding the LLMs to obtain good outputs [35]. We follow the existing works in diagram recognition using LLMs [23], [24], and engineer the prompt for the state diagrams as follows.

We anchored the prompt engineering with a single diagram, which in our case was Figure 1. We started with simple instructions and incrementally added more details to improve quality of the generated response. Our initial prompt was defined in a single sentence:

Try 1: `Infer the nodes and edges of the given graph.`

The LLM returned an empty list, indicating it could not complete the task due to the lack of guidance in the prompt about the expected answer. Therefore, we added more details regarding the output format:

Try 2: `Infer the nodes and edges of the given graph. The output should contain no text, but only edges in the form 'node1 -> node2'`

After the updated prompt, the LLM successfully generated a list of edges following the format. However, the response included dashes, so further refinement was necessary to remove the unexpected characters. We provided examples:

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.

Try 3: `Infer the nodes and edges of the given graph. The output should contain no text, but only edges in the form 'node1 -> node2'. For example:`
`node1 -> node2`
`node3 -> node4`

The quality of LLM answer improved notably. Yet, between some nodes, the edges were connected using `to` instead of `->`. We added an additional instruction to explicitly avert that:

Final: `Infer the nodes and edges of the given graph. The output should contain no text, but only edges in the form 'node1 -> node2'. For example:`
`node1 -> node2`
`node1 -> node1`
`Only use directed arrows: ->`

As the result, the LLM correctly identified the nodes and their relationships. We "froze" the prompt and used it throughout our study for all the diagrams.

B. Zero-shot Recognition of Diagrams

In the zero-shot setting, we do not provide the LLM with any additional context or examples. The LLM model is expected to generate the response based only on the initially provided instructions concatenated with the diagram image.

C. Few-shot Recognition of Diagrams

In the case of a few-shot recognition, we supply additional annotated examples ("shots") to the LLM so that it can "reason" not only over the prompt and pre-trained knowledge, but also over the concrete input-output examples, which can substantially increase the performance [35].

We propose to select the most complex diagrams as these few shots. While there are many different metrics for diagram complexity (*e.g.*, cyclomatic complexity [36]), we experimentally observed that simply counting the edges was satisfactory. Thus, we select a few diagrams with the largest number of edges to serve as examples, and manually annotate them. We then concatenated the images of the examples, their annotation, the image of the novel diagram and the instructions in the prompt. For example, we engineered the following prompt template for one-shot recognition:

Few-shot: `For the first diagram, we know that the edges are:`
`{annotated edges}`
`Infer the edges in the same format for the second diagram. Output only edges, no text.`

D. Code generation

For code-generation, we focus on an event-based architecture where the system is in exactly one state at a given time. A transition to another state along a connecting edge is then triggered by an event tied to the edge. The actual interpretation of an event depends on the use-case. It may be the reception of a network packet or a timeout which occurred. The developer has to provide an implementation of the `Event` class whose boilerplate code is shown in Listing 1.

```
struct Event
{
  // add custom members
};

Event wait_for_event()
{
  // fill appropriately
  return Event();
}
```

Listing 1: Boilerplate Event definition

For code generation, we aim at a template-based method to generate code in the C++ programming language. Given the output of the edge extraction, we generate three sections. For the first section, an `enum class` is generated containing an enumeration of system states. Additionally we output a helper function to stringify the enmeration value. An example is shown in Listing 2.

```
enum class State
{
  USB, WWW, ERP, JVM, CDN, LTE
};

std::string format_state(State const state)
{
  switch(state)
  {
    case State::USB:
      return "USB";
    // ...
  }
  return "?";
}
```

Listing 2: Generated Code: Enum section

In the second section, we generate a separate function for each state. This function handles the transition from this state to a new one after a certain `event` occurred. The actual handling of this transition has to be filled by the developer. The result is shown in Listing 3.

```
State handle_ERP(Event const & event)
{
  // Check the event and
  // return one of the following states:
  // - CDN
  // - USB
  // - WWW
  return State::ERP;
}
```

Listing 3: Handling of individual states in the generated code

Finally, the last section provides the entry point to our state handling boilerplate. It consists of a function handling each state transition and a `main` function. Note that fetching the next system state must also be implemented by the developer. The generated code is shown in Listing 4.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.

```
void handle_state(State const last_state,
                  State const new_state)
{
  switch(last_state)
  {
  case State::ERP:
      return handle_ERP(new_state);
  case State::JVM:
      return handle_JVM(new_state);
  // ...
  default:
    throw std::runtime_error(
      "Unknown state " +
      format_state(last_state));
  }
}
int main()
{
  State state = State::JVM;
  for(;;)
  {
    auto event = wait_for_event();
    state = handle_event(state, event);
  }
}
```

Listing 4: Generated Code: Entrypoint

The templates for each section can be modified by the developer. This enables simple customization, like specific namespaces or custom exceptions.

E. Test Case Generation

For test case generation, we rely on a coverage-based approach. We generate test cases for each graph separately. Here, a test case is defined by a path through the graph. By selecting a set of paths, *i.e.*, test cases, pre-defined coverage criteria can be met. One such criterion is to cover all edges in a graph. Take Figure 1 as an example. The following list of paths covers all edges in this graph:

- RSA → IPX → LVM → IPX → GUI → RSA
- RSA → IPX → LVM → IPX → RSA
- RSA → RSA

We used a modified depth-first search to generate such paths [37]. We can convert this to a set of unit tests by invoking the corresponding `handle` functions in order. An example of such an unit test is shown in Listing 5.

```
#include <gtest/gtest.h>
TEST(App, ERP)
{
  // CHeck path ['JVM', 'ERP', 'USB', 'LTE', 'JVM']
  Event event;
  auto state = State::JVM;
  // TODO: fill event appropriately
  state = handle_event(state, event);
  ASSERT_EQ(state, State::JVM);
  // TODO: fill event appropriately
  state = handle_event(state, event);
  ASSERT_EQ(state, State::ERP);
  // ...
  state = handle_event(state, event);
  ASSERT_EQ(state, State::JVM);
}
```

Listing 5: Generated unit test

IV. Experiments

To evaluate the effectiveness of our approach, we experimentally test it on two specifications of prominent Industry 4.0 communication protocols. All the following experiments have been performed in 3 trials to account for variability in LLM responses. We did not observe any noticeable difference between the responses, so we present here only the results from the first trial for better readability.

A. Datasets

1) Scrambling of diagrams: Due to copyright reasons, we preprocessed the diagrams before the experiments such that the text labels in the nodes were scrambled. This was necessary as we sent the data to cloud servers outside of our control. We ran an off-the-shelf OCR model [38], identified the text regions and replaced them with a random techno-babble abbreviations (such as ICQ, DNS or IOT). The scrambled diagrams thus closely resemble the original ones without revealing any copyrighted information.

2) IEC 61158 (PROFINET): Our main experiments focused on IEC 61158 which is a widely-read international standard for fieldbus communication systems in industrial automation [1]. It defines communication protocols used for real-time control and monitoring in industrial environments For our case study we selected the PROFINET communication protocol. PROFINET, developed by PROFIBUS & PROFINET International (PI), is based on Ethernet technology and provides high-speed and real-time communication for industrial automation and control systems. PROFINET is described in IEC 61158-6-10 which is provided in PDF format containing 1183 pages, from which we cropped in total 80 diagrams corresopnding to state machines. Diagrams are not overly complex neither in terms of node quantity (3.5±2.1 nodes on average, where only 2 diagrams had more than 7 nodes) nor in terms of edge quantity (8.2±6.6 edges on average, only 4 diagrams had more than 18 edges and the most complex one had 37). However, the diagrams are not straightforward to parse, For example, many edges are joined together, and even humans struggle with the crossings, see Figure 1.

3) IEC 62541 (OPC UA): We also source diagrams from another specification, IEC 62541, to see how our method generalizes. IEC 62541, also known as OPC UA, is an international standard developed by the OPC Foundation [39]. OPC UA is a communication protocol for industrial automation that ensures secure, reliable, and platform-independent data exchange between industrial devices and systems. The OPC UA specification consists of 24 parts provided in PDF format. The length of these parts vary between 11 and 268 pages resulting in 1830 pages total. We cropped 15 diagrams of state machines. In contrast to PROFINET dataset, the diagrams included edge labels and multiple edges between nodes, see Figure 3. Though informative for the reader, we considered both to be noise irrelevant for the code and test generation in our setting. The diagrams had on average more nodes (13.1±8.6 nodes, max. 29) and roughly equally many edges (8.5±5.3, max. 18) as IEC 61158 diagrams.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.

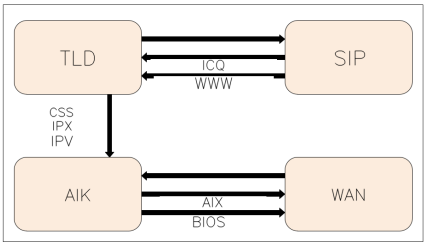
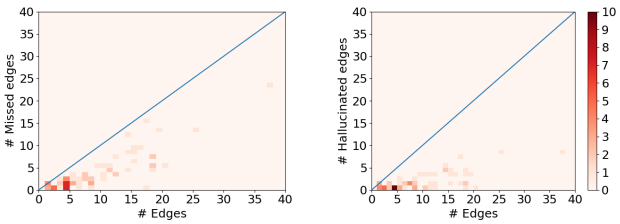


Fig. 3: State diagram from the IEC 62541 standard [39]

B. Zero-shot Recognition

1) Difference between models: We started first by evaluating different popular LLMs such as gpt-4o-2024-08-06 [17], claude-3-sonnet-20240229 [17] and and Llama-3.2-11b-vision-preview [18] on the zero-shot recognition of PROFINET diagrams to see how the performance differs between different models. We measured accuracy, *i.e.*, the fraction of ground truth edges which are also present in the response. We observe a notable difference between gpt-4o, claude 3 and Llaama 3: 68%, 40%, and 16%, respectively, indicating that the choice of the LLM is critical in practice. We used gpt-4o in the subsequent experiments given its far superior performance, and report only its performance for brevity.

2) Mistakes: We analyzed the performance of gpt-4o in more detail where we considered two kinds of mistakes: 1) missing edges (edges annotated in ground truth, but missing in the response) and 2) hallucinated edges (edges present in the response, but missing in the ground truth). Only 18 diagrams (out of 80) were recognized correctly, without any mistake. There were 53 diagrams with at most 3 mistakes. The number of mistakes expectedly grew with the number of edges in the diagram (*i.e.*, its complexity), as we see in Figure 4. Interestingly, the mistakes scaled sublinearly with the complexity, where edges are missed (Figure 4a) rather than hallucinated (Figure 4b).



a) Missing edges b) Hallucinated edges

Fig. 4: Distribution of mistakes in the LLM responses in relation to the number of edges in the ground truth: **a)** Expected edges missing in the response, **b)** Hallucinated edges unexpected in the response

The ground truth contains in total 657 edges. Overall, the LLM recognized 411 (63% of the ground truth), missed 246 (37% of the ground truth) and hallucinated 92 edges. These results indicate the amount of work saved by our approach: for 63% of edges, the operator is spared an action (insertion or deletion). There were negligible number of mistakes related to missing or hallucinating nodes (only 2 diagrams with one missing node and 4 diagrams with one hallucinated node). This is expected, as nodes are clearly discernible, while edges are much more imperceptible (*e.g.,* see Figure 1). The starting first node was successfully identified in all the diagrams (barring the one case of a hallucinated node). This is particularly important since it removes the tedium for the operator to specify the initial state in code and test generation.

Shots	Zero	One	Two	Three
Missed edges	245	**124**	129	130
Hallucinated edges	92	121	83	**78**
Total mistakes	337	245	212	**208**

TABLE I: Mistakes over number of shots on PROFINET

C. Few-shot Recognition

We tested with one, two and three shots. The results are shown in Table I. We observe that providing the examples to the LLM generally reduces the number of mistakes, and substantially so (from 337 without examples to 208 with three examples). However, providing more and more examples seems to "overwhelm" the model, as there is barely any difference in mistakes when two or three shots are given. Providing even a single example greatly reduces the missed edges (from 245 to 124), but leads to hallucinations (from 92 to 121). We speculate that a single example "primes" the model too much, which can be rectified by further examples.

D. Generalizability Across Specifications

We also evaluated on a different specification, OPC UA, to see how our diagram recognition generalizes. PROFINET and OPC UA have a significantly different appearance (*cf.* Figure 1 and Figure 3). In this experiment we thus tested for noise (edge labels, multiple edges between node pairs). The results are presented in Table II. We observe that supplying examples to the model did not help much: the model made fewest mistakes without any examples. This can be traced back to the complexity and diversity of OPC UA dataset. After inspection, we saw that edge labels cause hallucinations, and it seems that the model is overwhelmed by distinction between the node and edge labels. That a single example reduced the hallucinations (to only 14) can be considered rather a sampling artifact than a consistent behavior. The limitations of current LLMs pose also a limit on our approach, and OPC UA demonstrates such a case. However, we still note that 57 out of total 127 edges (45%) in the ground truth were correctly recognized without examples, roughly halving the necessary manual effort.

E. Code and Test Case Generation

We utilized the extracted state machines to generate code and unit test stubs as described in Subsection III-D and Subsection III-E. To get independent results we relied on

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.

Shots	Zero	One	Two	Three
Missed edges	**57**	121	60	64
Hallucinated edges	59	**14**	68	83
Total mistakes	**116**	135	128	147

TABLE II: Mistakes over number of shots on OPC UA

	Code	Tests
OPC UA	1614	1034
Profinet	7017	3934

TABLE III: Generated lines of code

the ground truth instead of using the actually extracted state machines. Table III summarizes the results. For OPC UA, we were able to generate 1614 lines of code and 1034 lines of unit tests, while for PROFINET, we generated 7017 lines of code and 3934 lines of test cases.

The generation process itself (excluding state machine extraction) took less than a second. This indicates a huge reduction in effort when compared to a manual workflow. Of course, a developer will need to review the code to correct mistakes in edge detection. However, the efficiency is improved drastically. Additionally, our approach ensures by construction that the generated unit tests cover the whole code base and are therefore of high quality.

F. Ablations

1) Diagram Recognition with Computer Vision: We evaluated a publicly-available state-of-the-art diagram recognition tool based on neural networks[2] on PROFINET dataset. Trained on business diagrams [14], vastly different from PROFINET diagrams, the tool consistently failed to recognize edges in diagrams like Figure 1. This corroborates that tools based on Computer Vision are not generalizable enough for general diagram recognition.

2) Direct Code and Test Generation from Diagrams: We explored using LLMs to generate code directly from diagrams, aiming to simplify recognition and generation into a single step. We used one-shot recognition and generation based on the following prompt:

```
For the given diagram, we know that the
source code looks like this: {source
code}. Infer the source code for the
second diagram. Output only the code, no
text.
```

We also evaluated the test code generation with one shot:

```
For the given diagram, we know that the
unit test code looks like this: {source
code}. Infer the source code for the
second diagram. Output only the code, no
text.
```

[2]https://sketch2bpmn.informatik.uni-mannheim.de/

Gpt-4o generated code which seems plausible at first glance. However, the LLM produces slight errors, which are difficult to spot. For example, some states are hallucinated in the generated code. Additionally, the mistakes in diagram recognition reported in Subsection IV-B, transpire to code and test generation. For test case generation, the LLM failed to meet the coverage criteria. This is somewhat expected, as LLMs cannot perform graph algorithms.

V. Discussion

Our experiments show that among state of the art LLMs, gpt-4o performs best. It is able to identify 63% and 45% of edges correctly for PROFINET and OPC UA, respecitvely. In this process, there are two kinds of mistakes: 1) either actually existing edges are missed by the LLM or 2) the LLM hallucinates edges which are not existing. Our experiments show that edges are rather missed than hallucinated. However, this can generally be improved by few-shot recognition with the trade-off that fixing some missing edges is off-set by generating more hallucinations. While both mistakes lead to buggy generation, and need to be manually fixed, it is highly application-dependent whether the one or the other causes more dangerous bugs. A missing edge definitely leads to non-conformant code, but a hallucinated edge leads to code potentially ending in unsafe or undefined states.

While the PROFINET specification contains mostly ”simple” state diagrams, the ones in the OPC UA specification are much more heterogeneous and contain special features like edge labels. As expected, the recognition rate drops from 63% to 45%. However, this is still a good result given that we did not feed an prior knowledge to the LLM. This highlights the high generalizability of LLMs. However, there is substantial room for improvement on the side of LLMs for this particular task, which we expect in the near future witnessing the current pace of research in LLMs. Since our approach is LLM-agnostic, we also benefit from their future advancements.

Finally, we evaluated code and unit test generation from the extracted state machines. We were able to successfully generate working C++ boilerplate code for both PROFINET and OPC UA. This saves a lot of manual effort as developers do not need to tediously write off the specification in code. Our template-based approach allows for easy customization like custom namespaces. Especially, it allows easy adoption for other programming languages and frameworks. We also generated stubs for unit tests based on coverage criteria in state machines. This ensures high code coverage and enforces test-driven development [40]. For comparison, we also evaluated direct code and test generation. However, results are mediocre, leaving much room for improvements in LLMs for this task. The generated code contains slight bugs, which are difficult to spot as the code looks plausible. Consequently, the generated code and tests are much more difficult to review than the list of edges, and require programming expertise. In contrast, reviewing recognized edges alone can be done by low-skilled operators, making our two-step approach more scalable.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusion and Future Work

In this work we presented an approach to generate code and unit tests from state diagrams extracted from specifications. First, we extract the images, then use an LLM to extract the state machines in machine-readible form. Finally, we use a template-based approach to generate code and unit tests from it. We conducted our experiments on the specifications of PROFINET and OPC UA, where the LLM (chatgpt-4o) was able to correctly extract 63% and 45% of the edges, respectively. We then leveraged these state diagrams to generate multiple thousands lines of C++ boilerplate code and unit test stubs. Our findings indicate a huge reduction of manual effort when implementing a specification.

In future work, we plan to infer nodes and edges in a two-step process — first nodes, then edges — to better handle complex diagrams. This approach is motivated by challenges in OPC UA, where dense labeling confused the LLM. Our current experiments used only two specifications; expanding this will enhance result expressiveness and reveal insights into LLM generalizability. Lastly, we aim to explore how computer vision techniques (*e.g.*, OCR, line detection) can help mitigate LLM misdetections.

References

[1] "IEC 61158: Industrial communication networks - fieldbus specifications," International Electrotechnical Commission (IEC), 2023, part 1 to Part 6: Overview, protocol-specific details, and guidelines.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.

[3] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3431–3440.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Neural Information Processing Systems (NeurIPS)*, 2017.

[5] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Learning Representations*, 2021.

[6] C. F. Moreno-García, E. Elyan, and C. Jayne, "New trends on digitisation of complex engineering drawings," *Neural Computing and Applications (NCAA)*, vol. 31, 2019.

[7] S. Mani, M. A. Haddad, D. Constantini, W. Douhard, Q. Li, and L. Poirier, "Automatic digitization of engineering diagrams using deep learning and graph search," in *Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020.

[8] R. Rahul, S. Paliwal, M. Sharma, and L. Vig, "Automatic information extraction from piping and instrumentation diagrams," in *Pattern Recognition Applications and Methods (ICPRAM)*, 2019.

[9] S. Kang, E.-B. Lee, and H.-K. Baek, "A digitization and conversion tool for imaged drawings to intelligent piping and instrumentation diagrams (P&ID)," *Energies*, vol. 12, no. 13, 2019.

[10] F. Chen, L. Zhang, X. Lian, and N. Niu, "Automatically recognizing the semantic elements from uml class diagram images," *Journal of Systems and Software*, vol. 193, 2022.

[11] B. Karasneh and M. R. Chaudron, "Extracting UML models from images," in *Computer Science and Information Technology*, 2013.

[12] A. Koenig, B. Allaert, and E. Renaux, "NEURAL-UML: Intelligent recognition system of structural elements in UML class diagram," in *Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2023.

[13] B. Schäfer, M. Keuper, and H. Stuckenschmidt, "Arrow R-CNN for handwritten diagram recognition," *Document Analysis and Recognition (IJDAR)*, vol. 24, no. 1, 2021.

[14] B. Schäfer, H. van der Aa, H. Leopold, and H. Stuckenschmidt, "Sketch2process: End-to-end BPMN sketch recognition based on neural networks," *Transactions on Software Engineering*, vol. 49, no. 4, 2023.

[15] B. Otto, R. Gröpler, K. Meinecke, and T. Kleinert, "From specification to test cases: A model-based approach using image recognition," 2023.

[16] R. Szeliski, *Computer Vision: Algoirhtms and Applications*. Springer Cham, 2022.

[17] A. Borji and M. Mohammadian, "Battle of the wordsmiths: Comparing chatgpt, gpt-4, claude, and bard," *SSRN*, 2025.

[18] A. Dubey *et al.*, "The llama 3 herd of models," *arXiv*, vol. abs/2407.21783, 2024.

[19] T. Brown and many other authors, "Language models are few-shot learners," in *Neural Information Processing Systems (NeurIPS)*, 2020.

[20] H. Koziolek, S. Grüner, R. Hark, V. Ashiwal, S. Linsbauer, and N. Eskandani, "LLM-based and retrieval-augmented control code generation," in *Large Language Models for Code (LLM4Code)*, 2024.

[21] H. Koziolek, S. Grüner, and V. Ashiwal, "ChatGPT for PLC/DCS control logic generation," in *Emerging Technologies and Factory Automation (ETFA)*, 2023.

[22] C. Tinnes, M. Ristin, U. Hohenstein, K. Fathi, and H. W. van de Venn, "From unstructured product descriptions to structured data for industry 4.0 with ChatGPT," in *Industrial Cyber-Physical Systems (ICPS)*, 2024.

[23] H. Koziolek and A. Koziolek, "LLM-based control code generation using image recognition," in *Large Language Models for Code (LLM4Code)*, 2024.

[24] A. Conrardy and J. Cabot, "From image to UML: First results of image based UML diagram generation using LLMs," in *Large Language Models for Model-Driven Engineering Workshop (LLM4MDE)*, 2024.

[25] B. Schätz, A. Pretschner, F. Huber, and J. Philipps, "Model-based development of embedded systems," in *International Conference on Object-Oriented Information Systems*. Springer, 2002, pp. 298–311.

[26] N. Braunisch, M. Ristin-Kaufmann, R. Lehmann, M. Wollschlaeger, and H. W. van de Venn, "Empowering industry 4.0 with generative and model-driven sdk development," in *IECON 2023-49th Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 2023, pp. 1–6.

[27] S. Rasnayaka, G. Wang, R. Shariffdeen, and G. N. Iyer, "An empirical study on usage and perceptions of llms in a software engineering project," in *Proceedings of the 1st International Workshop on Large Language Models for Code*, 2024, pp. 111–118.

[28] F. Lin, D. J. Kim *et al.*, "When llm-based code generation meets the software development process," *arXiv preprint arXiv:2403.15852*, 2024.

[29] L. Apfelbaum and J. Doyle, "Model based testing," in *Software quality week conference*. Citeseer, 1997, pp. 296–300.

[30] A. C. Dias Neto, R. Subramanyan, M. Vieira, and G. H. Travassos, "A survey on model-based testing approaches: a systematic review," in *Proceedings of the 1st ACM international workshop on Empirical assessment of software engineering languages and technologies: held in conjunction with the 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE) 2007*, 2007, pp. 31–36.

[31] B. Otto, K. Meinecke, and T. Kleinert, *Model-Based Test Case Generation for Compliance Checking of Reactive Asset Administration Shells*. Universitätsbibliothek der RWTH Aachen, 2022.

[32] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "Chatunitest: A framework for llm-based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 572–576.

[33] G. Ryan, S. Jain, M. Shang, S. Wang, X. Ma, M. K. Ramanathan, and B. Ray, "Code-aware prompting: A study of coverage-guided test generation in regression setting using llm," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 951–971, 2024.

[34] S. Eskenazi, P. Gomez-Krämer, and J.-M. Ogier, "A comprehensive survey of mostly textual document segmentation algorithms since 2008," *Pattern recognition*, vol. 64, pp. 1–14, 2017.

[35] T. Brown and many other authors, "Language models are few-shot learners," in *Neural Information Processing Systems (NeurIPS)*, 2020.

[36] T. J. McCabe, "A complexity measure," *Software Engineering*, vol. SE-2, no. 4, 1976.

[37] R. Sedgewick, *Algorithms in C, Part 5: Graph Algorithms*, 3rd ed. Addison Wesley Professional, 2001.

[38] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in *Computer Vision and Pattern Recognition (CVPR)*, 2019.

[39] *IEC 62541: OPC Unified Architecture - Specifications*, International Electrotechnical Commission (IEC) Std., 2020, available as parts 1–14 covering different aspects of the OPC UA standard.

[40] K. Beck, *Test driven development: By example*. Addison-Wesley Professional, 2022.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on March 04,2026 at 17:18:43 UTC from IEEE Xplore. Restrictions apply.